Task 1: Watch Terraform-04 video.
Task 2: Execute the script shown in the video.

Data Sources

1. Objective

To understand how Terraform uses a data source to read an existing file and uses a resource to create a new file with the same content.

2. Steps Used

1. Cleared the old Terraform configuration files and retained a backup of the Terraform state.
2. Created `dogs.txt` manually in the Terraform working directory with the content:

```
I love dogs than cats!
```

3. Created `main.tf` with the following configuration:

```
data "local_file" "dog" {
  filename = "${path.module}/dogs.txt"
}

resource "local_file" "my_pet" {
  filename = "${path.module}/pets.txt"
  content  = data.local_file.dog.content
}
```

4. Initialized the Terraform working directory using:

```
terraform init
```

5. Reviewed the planned changes using:

```
terraform plan
```

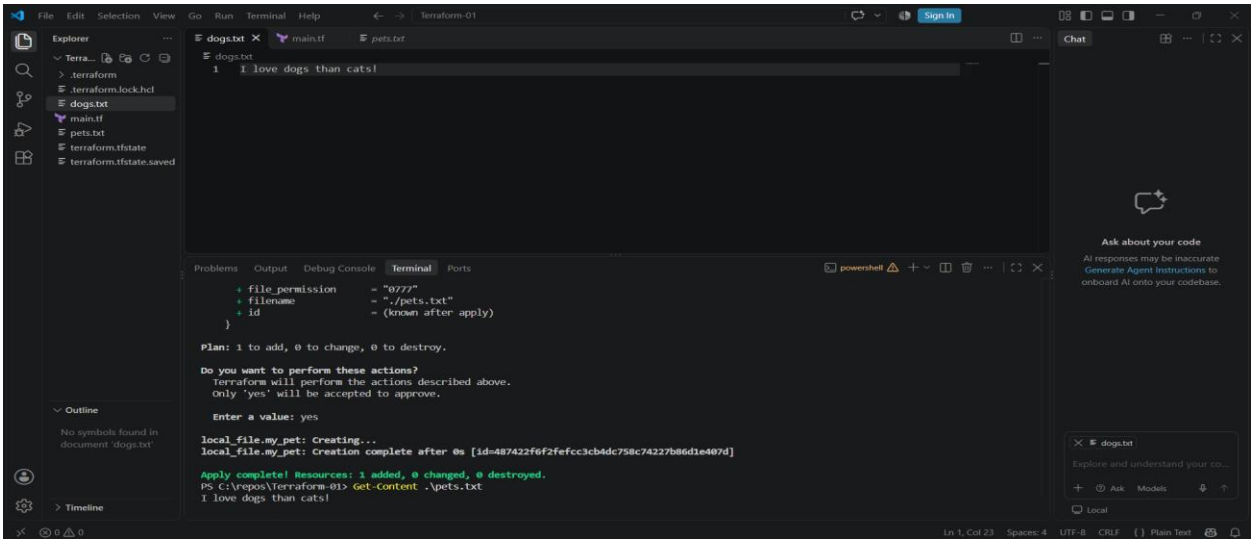
6. Executed the configuration using:

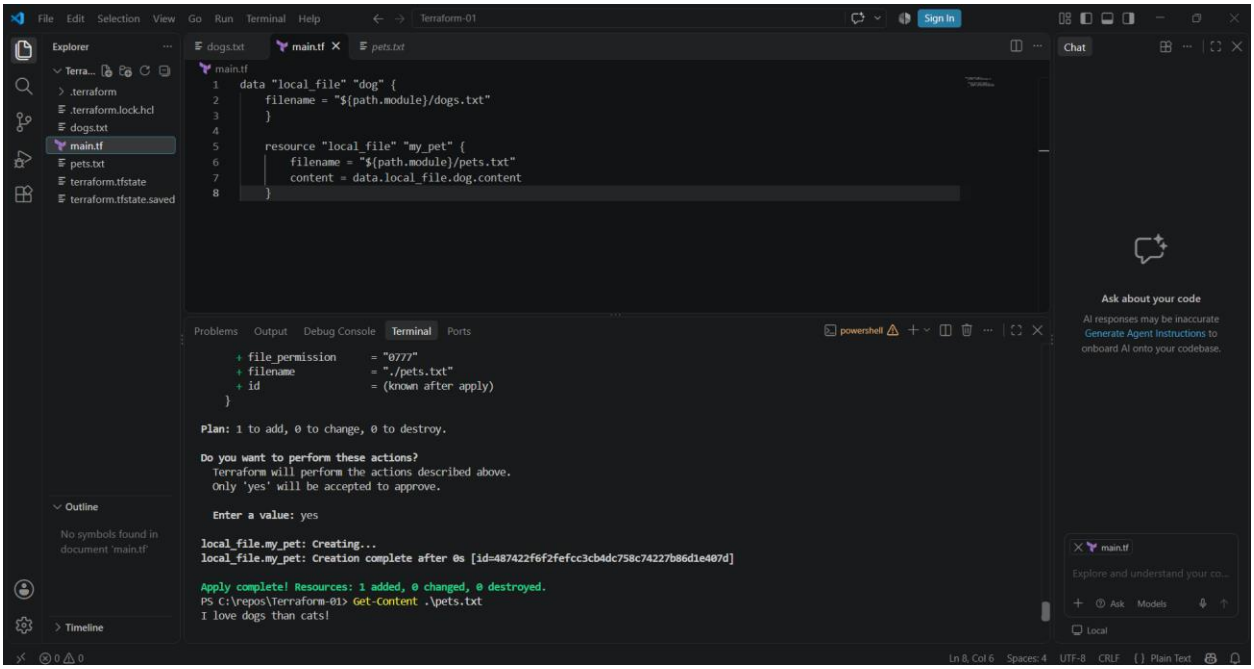
```
terraform apply
```

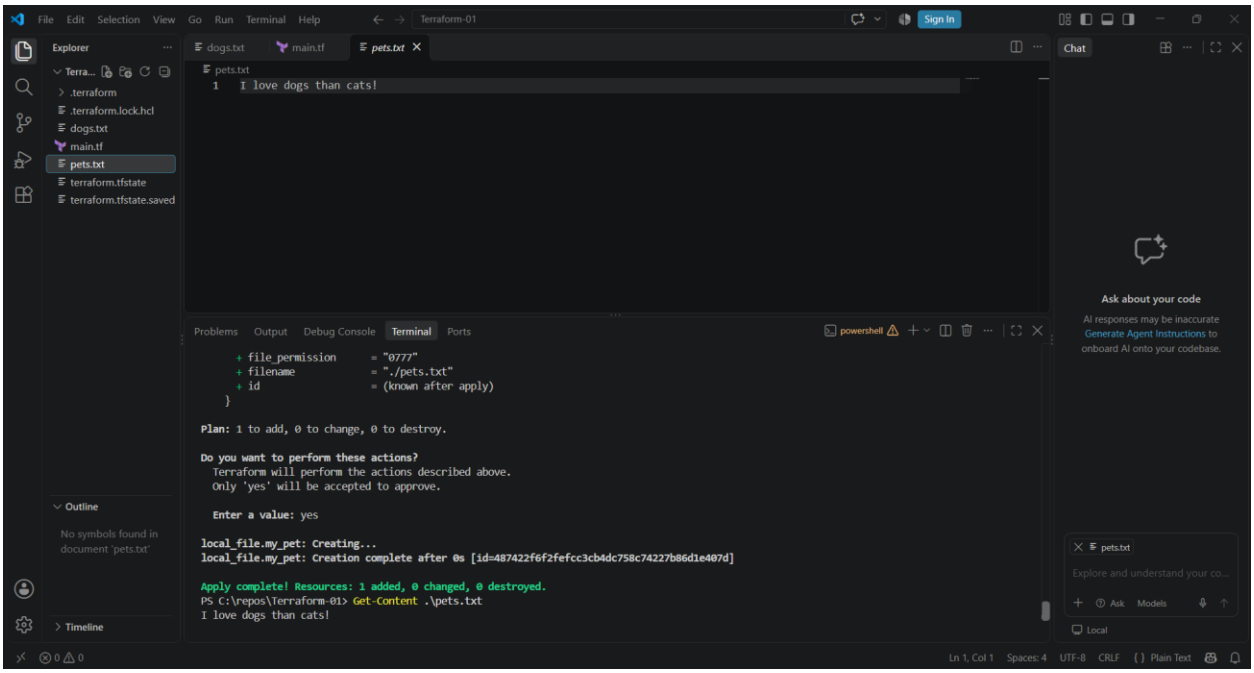
Confirmed the operation by entering `yes` when prompted.

7. Verified the contents of the generated file using:

```
Get-Content .\pets.txt
```







3. Observation

- Terraform successfully read the content of the existing `dogs.txt` file through the `local_file` data source.
- The resource `local_file.my_pet` used the content retrieved by the data source.
- Terraform planned to create one resource, with 1 to add, 0 to change, and 0 to destroy.
- The generated `pets.txt` file contained the same content as `dogs.txt`.

4. Conclusion

The lab demonstrated how Terraform data sources read information from existing objects and how resources use that information to manage new objects. The data source did not create or manage the lifecycle of `dogs.txt`; it only read its content.

5. Why do we use data sources?

We use data sources when Terraform needs information about something that already exists instead of creating it again.

Real-time DevOps example: An EC2 instance might already exist and have been created manually or by another automation tool. Terraform can use an AWS data source to retrieve its

details, such as its instance ID, availability zone, or IP address, and use those details in another resource's configuration.

Common use cases:

- Reading existing AWS infrastructure.
- Reusing information created by another Terraform configuration or tool.
- Retrieving existing file contents or configuration details.
- Avoiding the unnecessary recreation of existing infrastructure.

Key difference: A resource manages an object's lifecycle; a data source reads information about an existing object.

`count` Meta-Argument

1. Objective

To understand how Terraform's `count` meta-argument creates multiple instances of the same resource using a single resource block and how `count.index` assigns a unique index to each instance.

2. Steps Used

1. Updated `main.tf` with the following configuration:

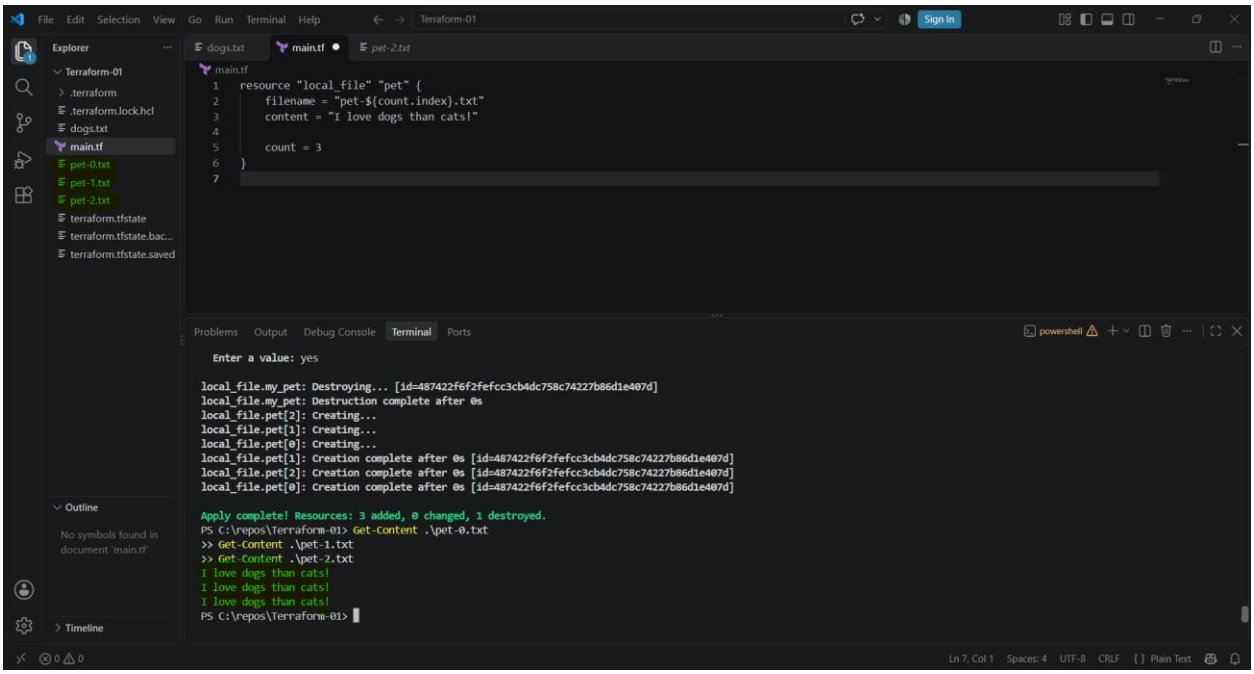
```
resource "local_file" "pet" {
  filename = "pet-${count.index}.txt"
  content  = "I love dogs than cats!"

  count = 3
}
```

2. Executed `terraform plan` to preview the changes.
3. Reviewed the plan, which showed 3 resources to add and 1 resource to destroy. The previous `local_file.my_pet` resource was no longer present in the configuration.
4. Executed `terraform apply` and entered `yes` to approve the changes.
5. Verified the created files using:

```
Get-Content .\pet-0.txt
Get-Content .\pet-1.txt
Get-Content .\pet-2.txt
```

6. Confirmed that all three files contained the expected content: `I love dogs than cats!`.



3. Observation

- Terraform created three files: `pet-0.txt`, `pet-1.txt`, and `pet-2.txt`.
- The `count` meta-argument created three instances from one resource block.
- The `count.index` expression generated indexes starting at `0`.
- Terraform destroyed the previously managed `pets.txt` resource because its resource block had been removed from the configuration.

4. Conclusion

The lab demonstrated how to use the `count` meta-argument to create multiple resources efficiently without writing separate resource blocks for each file.

5. Why do we use `count`?

We use `count` when we need to create a specific number of similar resources.

Real-time DevOps examples:

- Creating multiple EC2 instances for a development environment.
- Creating multiple files or similar resources from one configuration.
- Creating a fixed number of test servers.

Important limitation

When `count` is used with a list, Terraform identifies instances by numeric indexes, such as `resource.example[0]` and `resource.example[1]`. If list elements are removed or reordered, the index-to-item mapping can change, potentially causing unnecessary updates or replacements.

When to use which:

- `count` — best when you need a fixed number of similar instances or index-based instances.
- `for_each` — best when each resource should have a stable identity based on a key, such as a filename or server name.

Terraform `for_each`

1. Objective

Create multiple files using a single Terraform resource block and the `for_each` meta-argument.

2. Terraform Code (`main.tf`)

```
variable "filename" {
  type = set(string)

  default = [
    "pets.txt",
    "cats.txt",
```

```
    "dogs.txt"
  ]
}

resource "local_file" "pet" {
  filename = each.value
  content  = "I love dogs than cats!"

  for_each = var.filename
}
```

3. Code Explanation

Code	Meaning
`variable "filename"`	Defines a variable named `filename`.
`type = set(string)`	Accepts a collection of unique strings.
`default`	Specifies the three filenames.
`resource "local_file" "pet"`	Defines a resource to create files.
`filename = each.value`	Uses the current filename from the collection.
`content`	Defines the content written into each file.
`for_each = var.filename`	Creates one resource instance for each value.

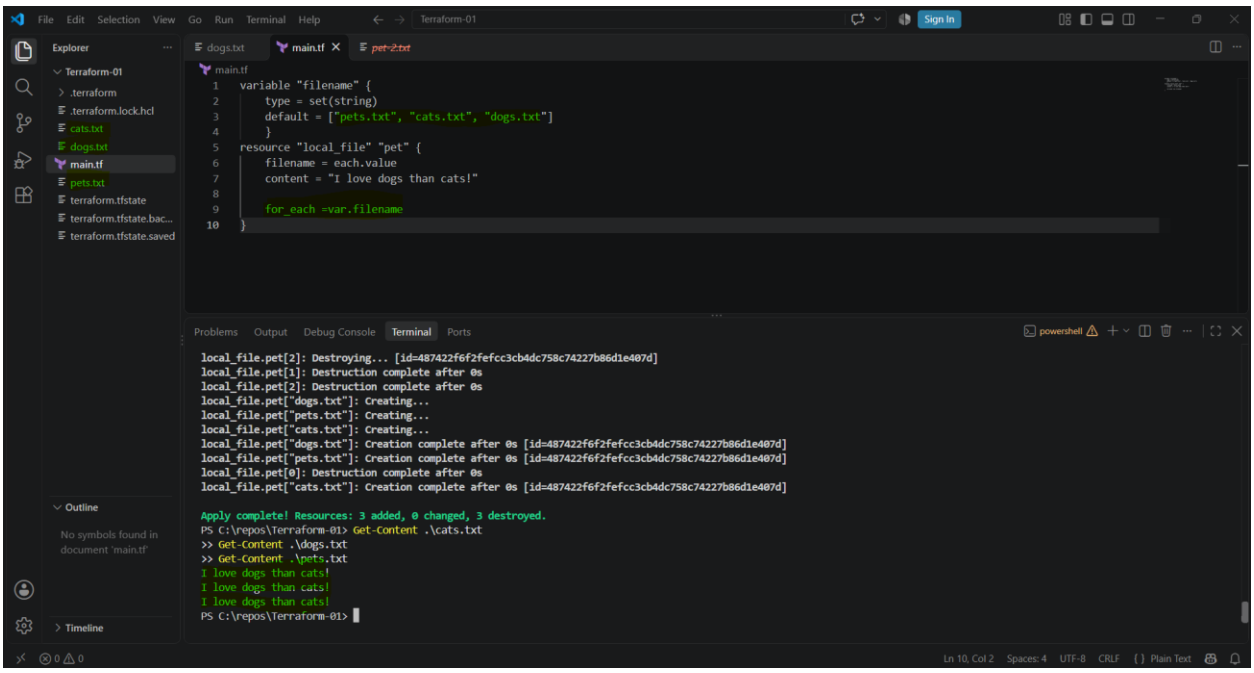
4. Commands Used

```
terraform init
terraform plan
terraform apply
```

Type `yes` when Terraform asks for approval.

Verify the files:

```
Get-Content .\cats.txt
Get-Content .\dogs.txt
Get-Content .\pets.txt
```



5. Observation

Terraform created three files:

- `cats.txt`
- `dogs.txt`
- `pets.txt`

Each contains `I love dogs than cats!`. The previous `count`-based instances were destroyed because Terraform now identifies resources using `for_each` keys instead of numeric indexes.

6. Conclusion

The `for_each` meta-argument allows Terraform to create multiple resources from a collection. Each instance has a meaningful key, making it useful when managing resources with distinct names or stable identifiers.

Why use it?

It is useful when managing resources with distinct names or stable identifiers, such as AWS resources for different applications or environments.

Difference: `count` identifies instances by numeric indexes (`0`, `1`, `2`), while `for_each` identifies them by keys (`"cats.txt"`, `"dogs.txt"`, `"pets.txt"`).

Terraform Version Constraints — Documentation

1. Objective

To specify the Terraform CLI versions allowed to run a Terraform configuration and ensure compatibility across development and deployment environments.

2. Steps Used

Step 1: Configure the version constraint

Added the following block to `main.tf`:

```
terraform {
  required_version = ">= 1.0, < 2.0"
}
```

Step 2: Check the installed Terraform version

```
terraform version
```

Output showed Terraform `v1.17.0` on `windows_amd64`.

Step 3: Validate the configuration

```
terraform validate
```

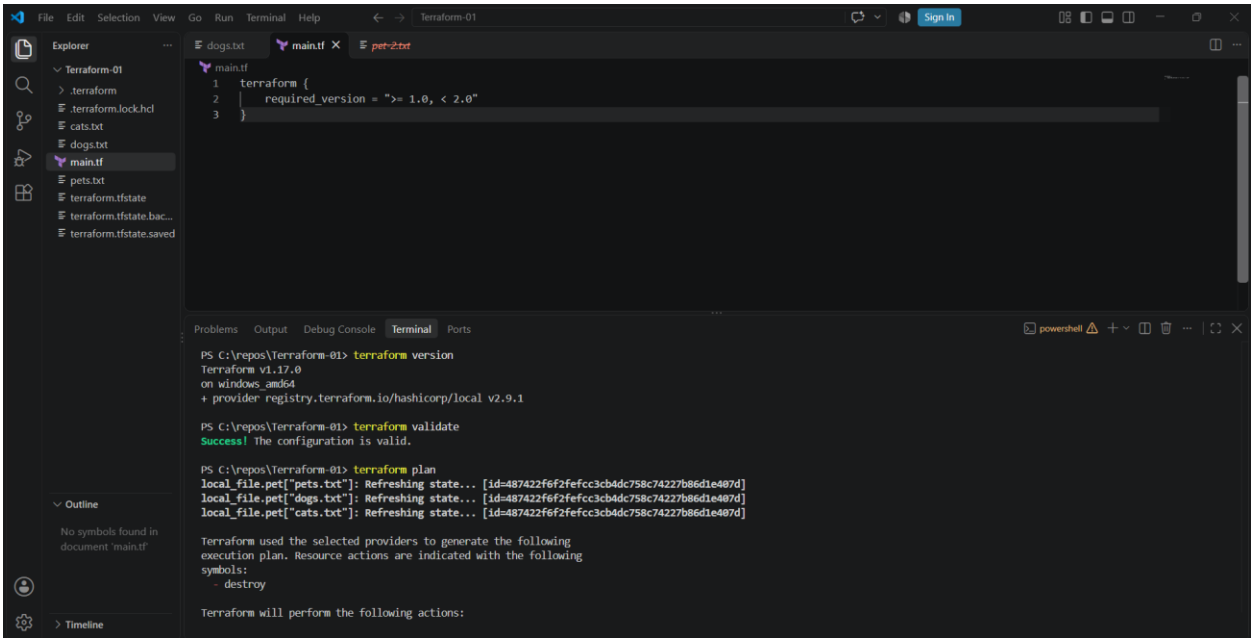
Output:

```
Success! The configuration is valid.
```

Step 4: Review the execution plan

```
terraform plan
```

The plan showed `0 to add, 0 to change, 3 to destroy` because the existing `local_file.pet` resource was missing from the current configuration. No changes were applied.



3. Observation

- The installed Terraform version, `1.17.0`, satisfies the constraint `>= 1.0, < 2.0`.
- Terraform validation succeeded.
- The plan detected three resources to destroy because their resource block was absent from `main.tf`. This was unrelated to the version constraint.

4. Conclusion

Terraform version constraints define which CLI versions are compatible with a configuration. The constraint used in this lab permits Terraform versions from `1.0` up to, but not including, `2.0`.

5. Why Do We Use Version Constraints?

- Version compatibility: Prevents running configurations with unsupported Terraform versions.
- Team consistency: Helps developers use compatible versions across their systems.
- CI/CD reliability: Reduces unexpected pipeline failures caused by Terraform version differences.
- Controlled upgrades: Helps teams avoid automatically adopting a major Terraform version that may introduce breaking changes.

Real-time example: If a company tests its infrastructure code with Terraform 1.x, it can restrict the configuration to versions below 2.0 until the team has tested and approved a migration.

Terraform IAM User Creation

1. Objective

To create an AWS IAM user named `asif` using Terraform in VS Code, understand Terraform resource management, and configure AWS credentials to connect Terraform with AWS.

2. Steps Used

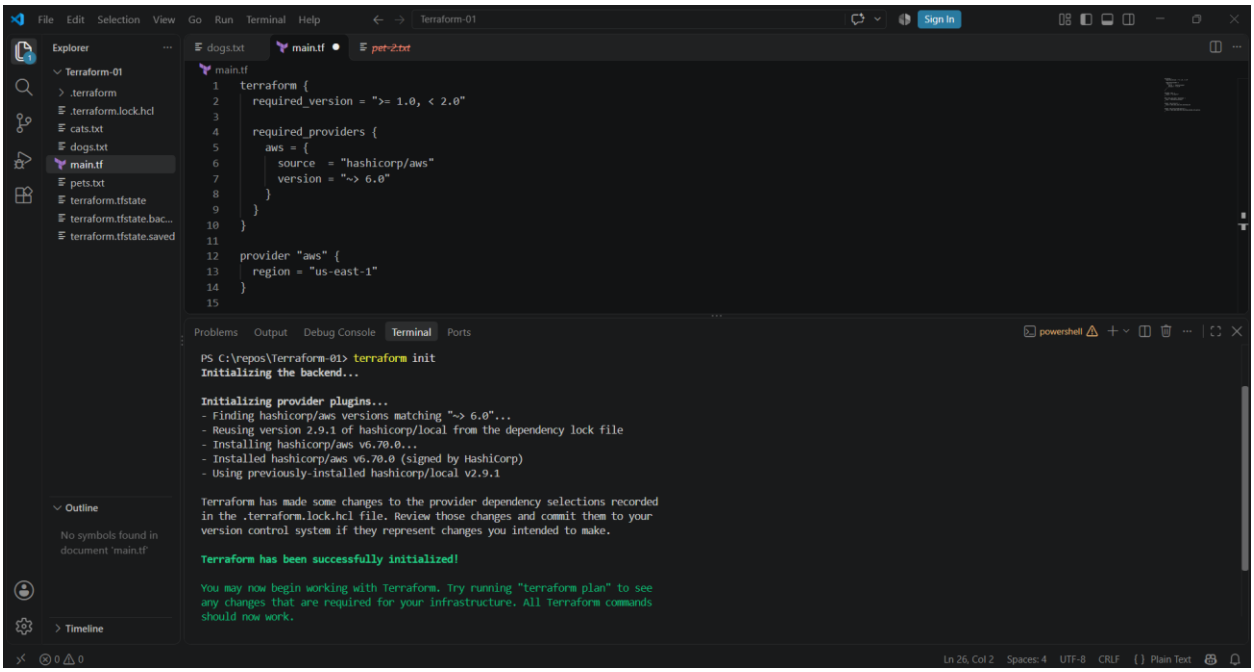
1. Created a project folder named `Terraform-IAM` in `C:\repos\Terraform-IAM`.
2. Created a `main.tf` file in VS Code.
3. Configured the Terraform AWS provider using AWS provider version `~> 6.0`.
4. Specified the AWS region as `us-east-1` (N. Virginia).
5. Configured the AWS profile as `terraform-bootstrap` for authentication.
6. Defined an IAM user resource in Terraform:

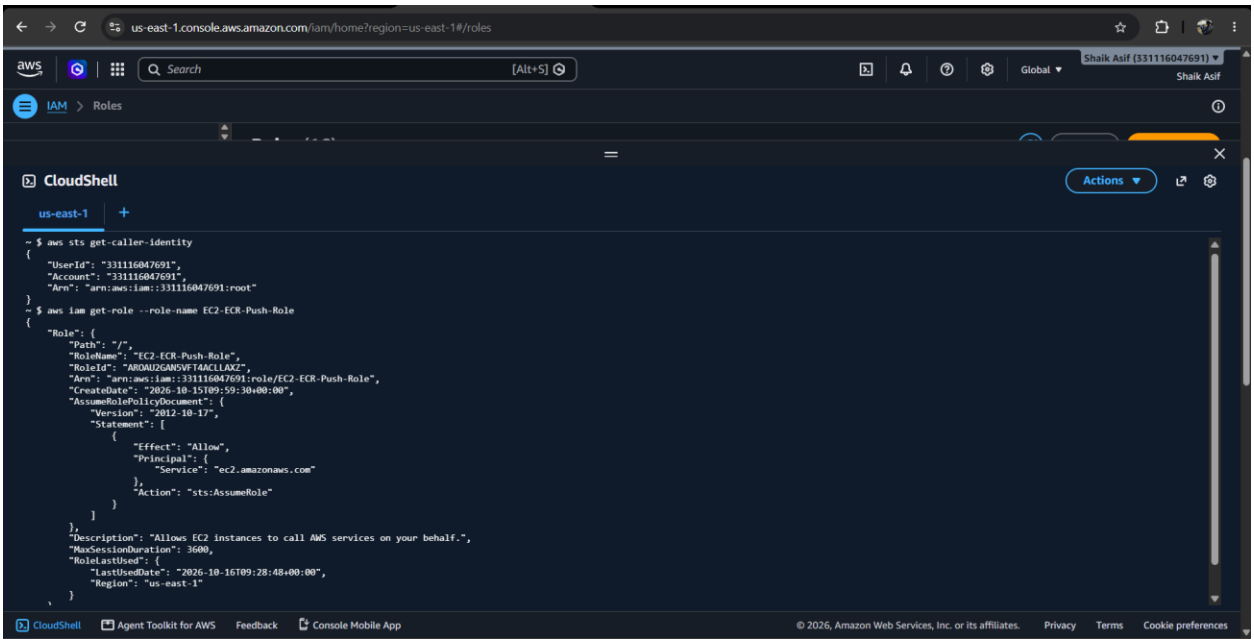
```
resource "aws_iam_user" "practice" {
  name = "asif"
}
```

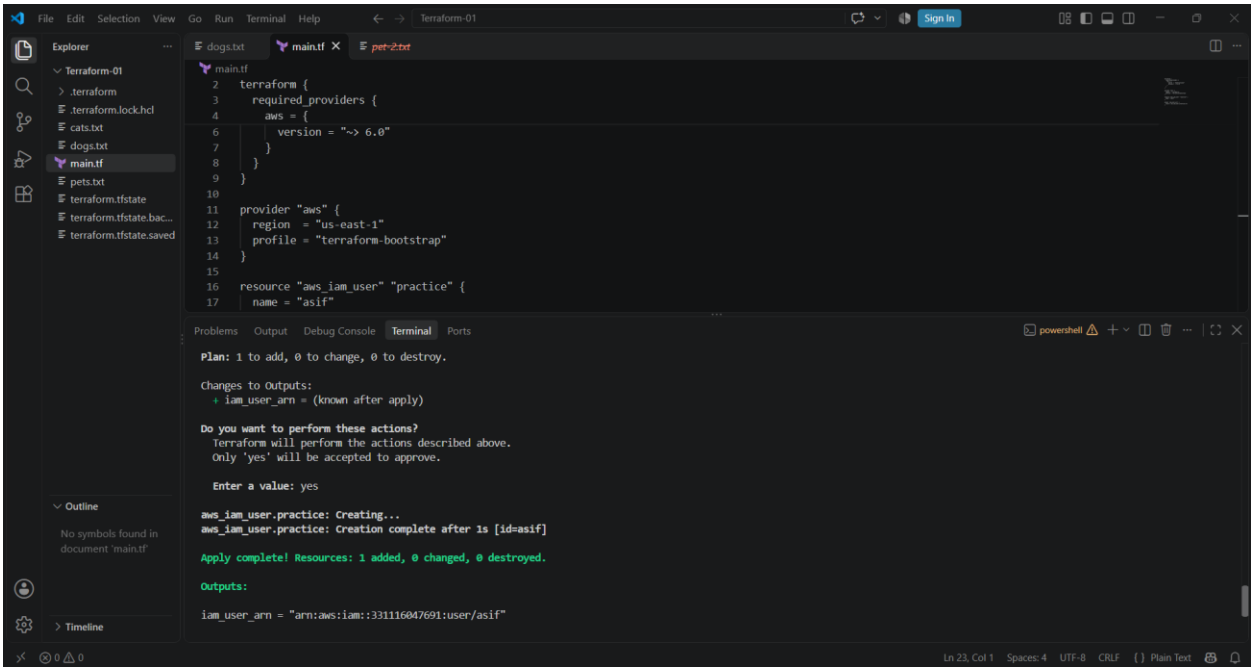
7. Added an output block to display the IAM user's ARN:

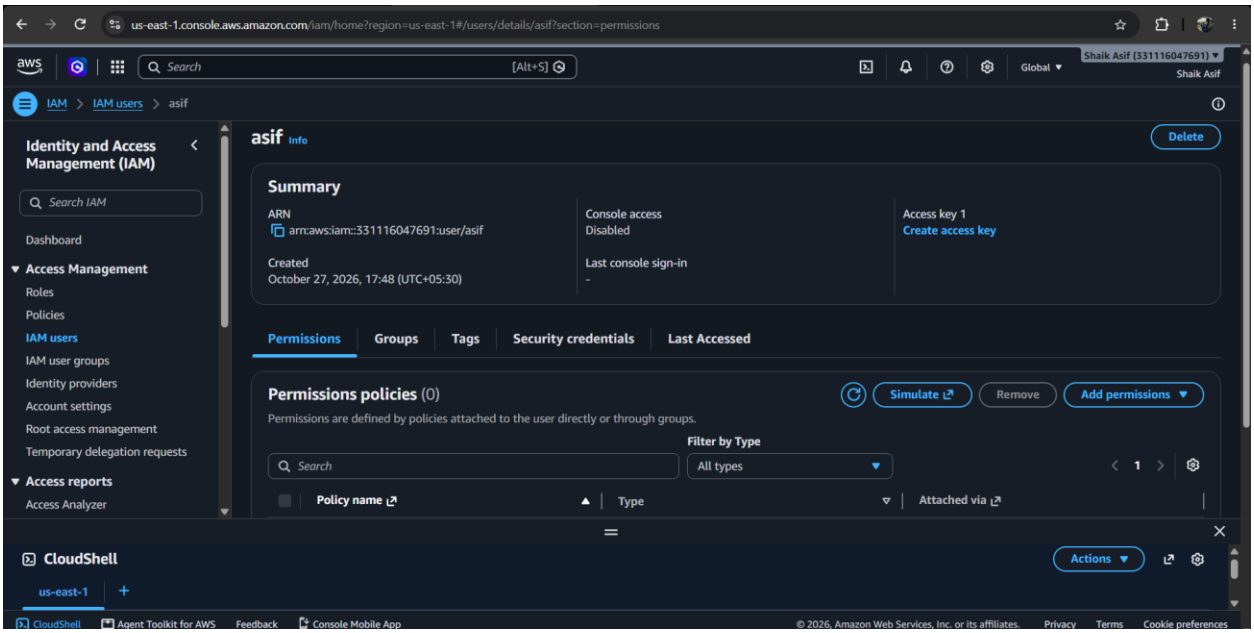
```
output "iam_user_arn" {
  value = aws_iam_user.practice.arn
}
```

8. Initialized Terraform using `terraform init`.
9. Executed `terraform plan` to review the proposed infrastructure changes.
10. Applied the Terraform configuration to create the IAM user `asif`.
11. Verified the Terraform state using `terraform state list`.




us-east-1.console.aws.amazon.com/iam/home?region=us-east-1#/roles
Search
[Alt+S]
Global
Shaik Asif (331116047691)
Shaik Asif
IAM > Roles
CloudShell
Actions
us-east-1
~ $ aws sts get-caller-identity
{
"UserId": "331116047691",
"Account": "331116047691",
"Arn": "arn:aws:iam::331116047691:root"
}
~ $ aws iam get-role --role-name EC2-ECR-Push-Role
{
"Role": {
"Path": "/",
"RoleName": "EC2-ECR-Push-Role",
"RoleId": "AROAU2GAN5VFT4ACLLAXZ",
"Arn": "arn:aws:iam::331116047691:role/EC2-ECR-Push-Role",
"CreateDate": "2026-10-15T09:59:30+00:00",
"AssumeRolePolicyDocument": {
"Version": "2012-10-17",
"Statement": [
{
"Effect": "Allow",
"Principal": {
"Service": "ec2.amazonaws.com"
},
"Action": "sts:AssumeRole"
}
]
},
"Description": "Allows EC2 instances to call AWS services on your behalf.",
"MaxSessionDuration": 3600,
"RoleLastUsed": {
"LastUsedDate": "2026-10-16T09:28:48+00:00",
"Region": "us-east-1"
}
}
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


File Edit Selection View Go Run Terminal Help
Terraform-01
Sign In
Explorer
Terraform-01
.terraform
.terraform.lock.hcl
cats.txt
dogs.txt
main.tf
pets.txt
terraform.tfstate
terraform.tfstate.bac...
terraform.tfstate.saved
dogs.txt
main.tf
pet-2.txt
main.tf
2 terraform {
3 required_providers {
4 aws = {
6 version = "~> 6.0"
7 }
8 }
9 }
10
11 provider "aws" {
12 region = "us-east-1"
13 profile = "terraform-bootstrap"
14 }
15
16 resource "aws_iam_user" "practice" {
17 name = "asif"
Problems
Output
Debug Console
Terminal
Ports
powershell
Plan: 1 to add, 0 to change, 0 to destroy.
Changes to Outputs:
+ iam_user_arn = (known after apply)
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
aws_iam_user.practice: Creating...
aws_iam_user.practice: Creation complete after 1s [id=asif]
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
Outputs:
iam_user_arn = "arn:aws:iam::331116047691:user/asif"
Outline
No symbols found in document 'main.tf'
Timeline
Ln 23, Col 1
Spaces: 4
UTF-8
CRLF
Plain Text



3. Observation

- Terraform successfully created the IAM user `asif`.
- The IAM user ARN was displayed as:

`arn:aws:iam::331116047691:user/asif`

- Terraform tracked the resource in its state file as:

`aws_iam_user.practice`

- The `terraform state list` command confirmed that the IAM user was recorded in Terraform state.
- An attempt to add the AWS `ReadOnlyAccess` managed policy attachment was unsuccessful because the saved configuration did not contain the attachment resource when checked.
- AWS credential configuration was also explored. The credentials verified by `aws sts get-caller-identity` belonged to the root account, highlighting the importance of using a dedicated IAM identity for Terraform operations.

4. Conclusion

This exercise demonstrated how Terraform can create and manage an AWS IAM user through Infrastructure as Code (IaC). It covered provider configuration, resource definition, output values, initialization, planning, and state verification.

The IAM user `asif` was created successfully.

Task 3: Integrate Terraform in Jenkins using the Terraform plugin.

1. Objective

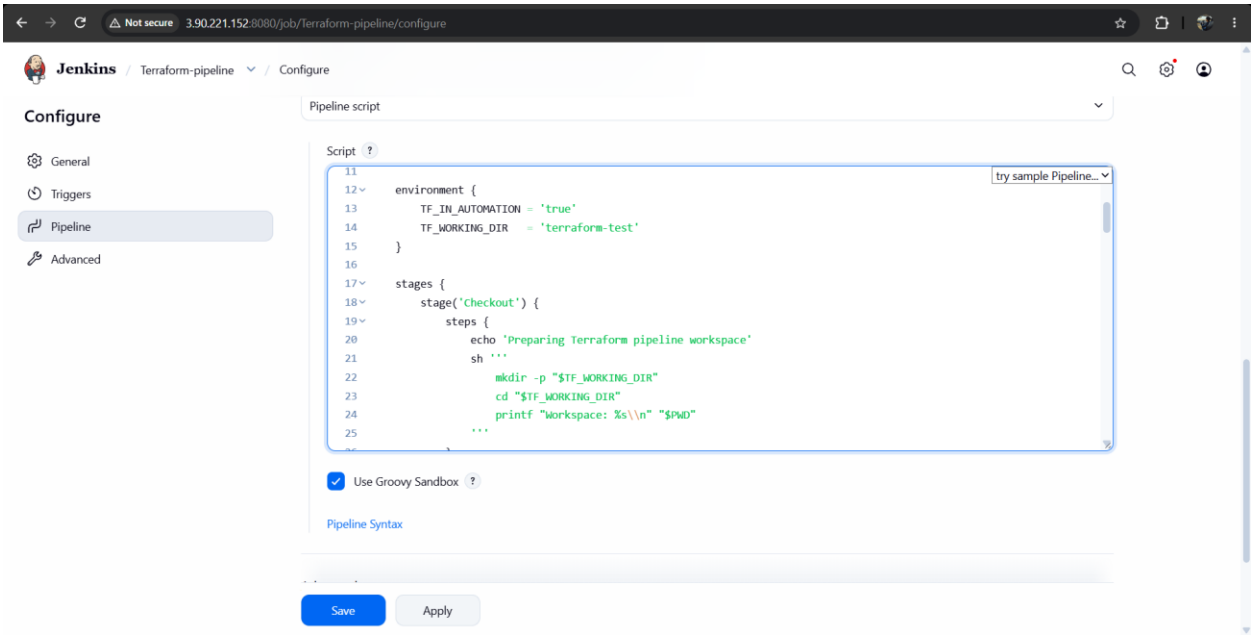
To integrate Terraform with Jenkins using the Terraform plugin and create an automated pipeline to execute Terraform commands, validate configuration, generate an execution plan, apply changes, and display outputs without manual approval.

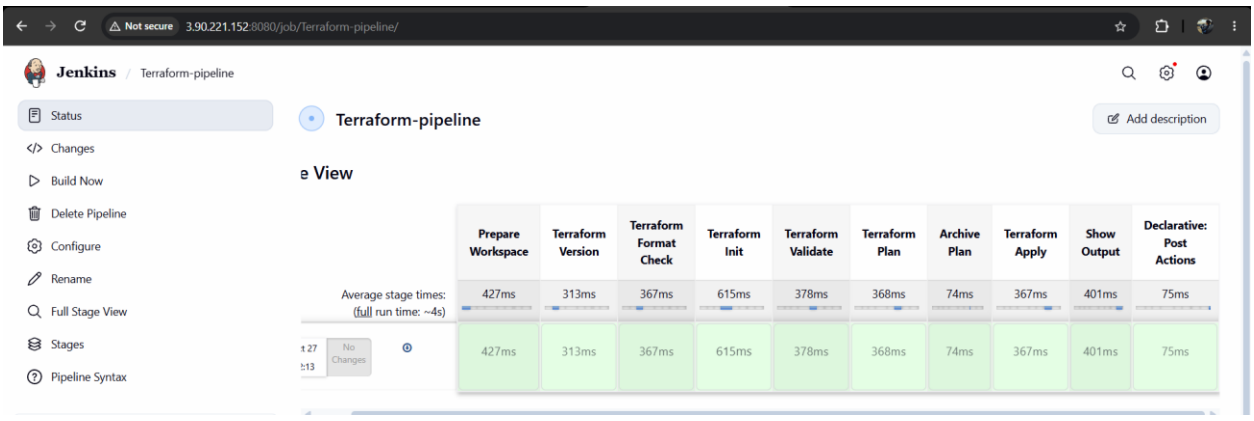
2. Steps Used

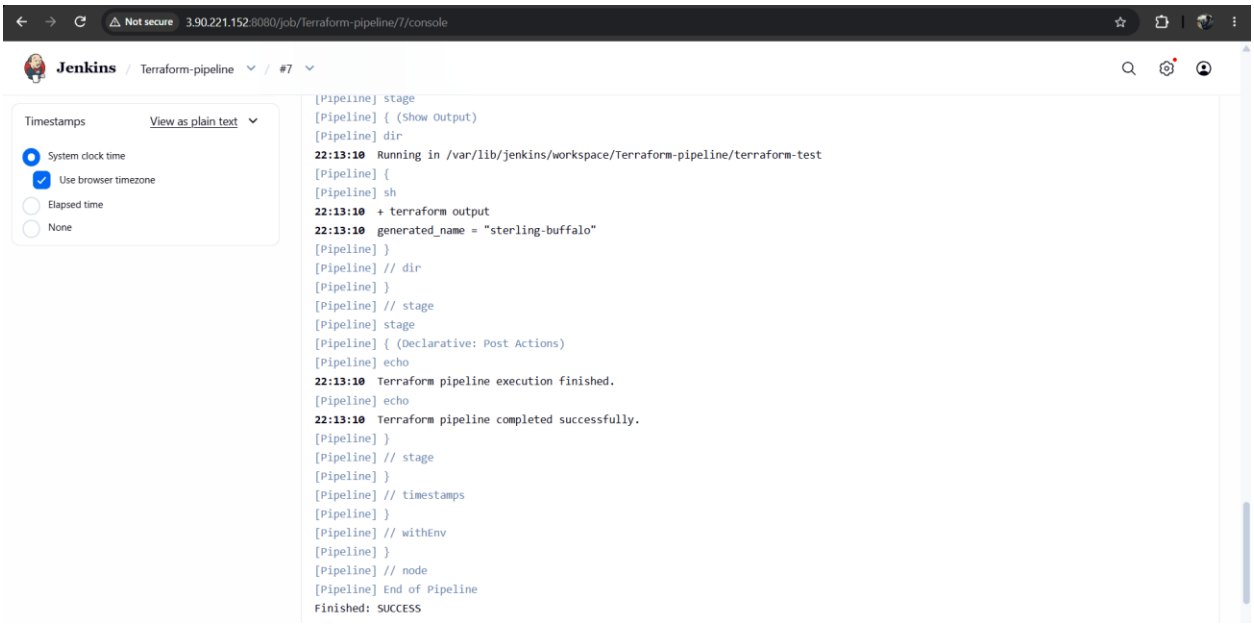
1. Opened the Jenkins dashboard.
2. Verified that the Terraform plugin was installed under Manage Jenkins → Plugins → Installed plugins.
3. Verified Terraform configuration under Manage Jenkins → Tools.
4. Opened the existing `Terraform-pipeline` job and selected Configure.
5. Selected the Pipeline script configuration.
6. Created a declarative Jenkins pipeline with the following stages:
    - Prepare Workspace: Created a dedicated `terraform-test` directory and generated `main.tf`.
    - Terraform Version: Verified the installed Terraform version using `terraform version`.
    - Terraform Format Check: Checked Terraform formatting using `terraform fmt -check -recursive`.
    - Terraform Init: Initialized the Terraform working directory and downloaded the required provider.
    - Terraform Validate: Checked the Terraform configuration for syntax and configuration errors.
    - Terraform Plan: Generated an execution plan using `terraform plan -input=false -out=tfplan`.

- Archive Plan: Archived the Terraform plan and configuration as Jenkins build artifacts.
- Terraform Apply: Applied the saved plan automatically using `terraform apply -input=false -auto-approve tfplan`.
- Show Output: Displayed the generated Terraform output using `terraform output`.

7. Removed the manual approval stage so the pipeline could proceed automatically.
8. Saved the pipeline and executed it using Build Now.
9. Reviewed the build stages and Console Output to verify execution.







3. Observation

- Jenkins successfully executed Terraform commands.
- Terraform version `1.17.0` was available on the Jenkins server.
- The initial pipeline failed during the Plan stage because the working directory did not contain Terraform configuration files.
- The pipeline was updated to generate a `main.tf` file using the `random_pet` resource.
- The updated pipeline was designed to run initialization, validation, planning, application, and output stages automatically.
- The `random_pet` resource generates a random name stored in Terraform state; this demonstration does not create AWS infrastructure.

4. Conclusion

Terraform was integrated with Jenkins through a declarative pipeline, demonstrating Infrastructure as Code automation within a CI/CD workflow. The pipeline includes formatting, initialization, validation, planning, artifact archiving, automatic application, and output verification.

The exercise demonstrated how Jenkins can automate Terraform operations without manual intervention. The final successful execution should be confirmed from the Jenkins build result and Console Output.

Task 4: Create a CI/CD pipeline for a Nodejs Application:

github.com/betawins/Trading-UI.git

1. Objective

To automate the build and deployment of a React application using Jenkins CI/CD. The pipeline checks out source code from GitHub, installs dependencies, builds the React application, deploys the production files on an AWS EC2 instance, and verifies application availability on TCP port 6558.

2. Tools and Technologies Used

- GitHub: Source code repository
- Jenkins: CI/CD pipeline automation
- AWS EC2: Application hosting server
- Node.js and npm: JavaScript runtime and dependency management
- React: Frontend application framework
- RxJS: JavaScript reactive programming library
- serve: Static web server for the production build
- Linux shell commands: Build and deployment automation

3. Steps Used

1. Configured the Jenkins job: Created the `Trading-UI-CICD` pipeline job.
2. Checked out source code: Downloaded the React application from the GitHub `master` branch.
3. Installed dependencies: Verified Node.js and npm, removed the existing `node_modules` directory, verified the npm cache, and installed project dependencies.
4. Resolved the RxJS issue: Installed RxJS version `6.6.7` as a workaround for the missing `canReportError` module.
5. Resolved the OpenSSL compatibility issue: Configured `NODE_OPTIONS=--openssl-legacy-provider` for the older Webpack build.
6. Built the React application: Executed `CI=false npm run build` and verified that `build/index.html` was generated.
7. Prepared the deployment directory: Created `/var/lib/jenkins/asif-trading-ui-deploy`.
8. Deployed the application: Copied the production build files to the deployment directory.

9. Started the web server: Used `serve` to host the React production build on TCP port `6558`.
10. Verified the deployment: Used `curl` to check the local application endpoint. Finally, opened the application through the browser.

4. Jenkins Pipeline Used

```
pipeline {
    agent any

    environment {
        APP_NAME    = 'Asif-Trading-UI'
        APP_PORT    = '6558'
        DEPLOY_DIR  = '/var/lib/jenkins/asif-trading-ui-deploy'
        NODE_OPTIONS = '--openssl-legacy-provider'
    }

    stages {
        stage('Checkout') {
            steps {
                git branch: 'master',
                    url: 'https://github.com/betawins/Trading-UI.git'
            }
        }

        stage('Install Dependencies') {
            steps {
                sh '''
                    set -e
                    node --version
                    npm --version

                    rm -rf node_modules
                    npm cache verify

                    npm install --legacy-peer-deps --omit=optional
                    npm install rxjs@6.6.7 \
                        --no-save \
```

```
                    --legacy-peer-deps \
                    --omit=optional

                node -e "require('rxjs'); console.log('RxJS loaded
successfully')"
            '''
        }
    }

    stage('Build') {
        steps {
            sh '''
                set -e

                CI=false npm run build
                test -f build/index.html

                echo "React production build completed."
            '''
        }
    }

    stage('Deploy') {
        steps {
            sh '''
                set -e

                mkdir -p "$DEPLOY_DIR"
                cp -r build/. "$DEPLOY_DIR/"

                if [ -f "$DEPLOY_DIR/serve.pid" ]; then
                    OLD_PID=$(cat "$DEPLOY_DIR/serve.pid")

                    if kill -0 "$OLD_PID" 2>/dev/null; then
                        kill "$OLD_PID" || true
                        sleep 2
                    fi
                fi
```

```
                    if command -v fuser >/dev/null 2>&1; then
                        fuser -k "${APP_PORT}/tcp" 2>/dev/null || true
                        sleep 2
                    fi

                    cd "$DEPLOY_DIR"

                    JENKINS_NODE_COOKIE=dontKillMe \
                    nohup serve -s "$DEPLOY_DIR" -l "$APP_PORT" \
                        > "$DEPLOY_DIR/serve.log" 2>&1 &

                    echo $! > "$DEPLOY_DIR/serve.pid"

                    sleep 3

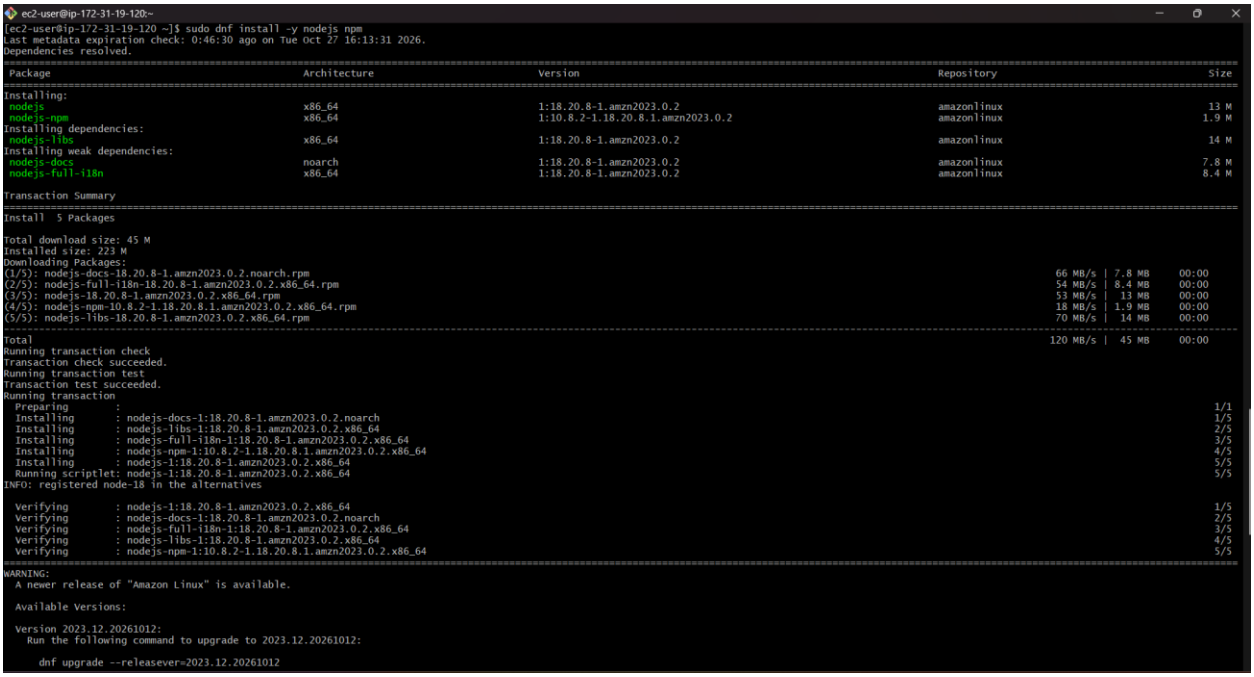
                    curl --retry 5 \
                        --retry-delay 2 \
                        --retry-connrefused \
                        -fsS "http://localhost:${APP_PORT}/" \
                        -o /dev/null

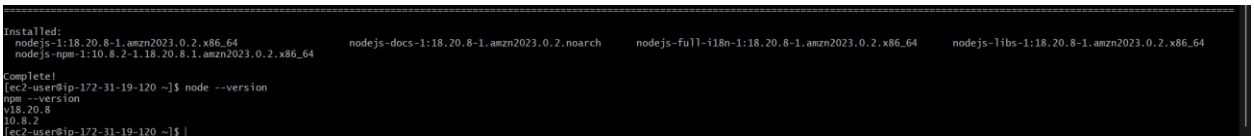
                    echo "${APP_NAME} deployed successfully on port
${APP_PORT}."
                '''
            }
        }
    }

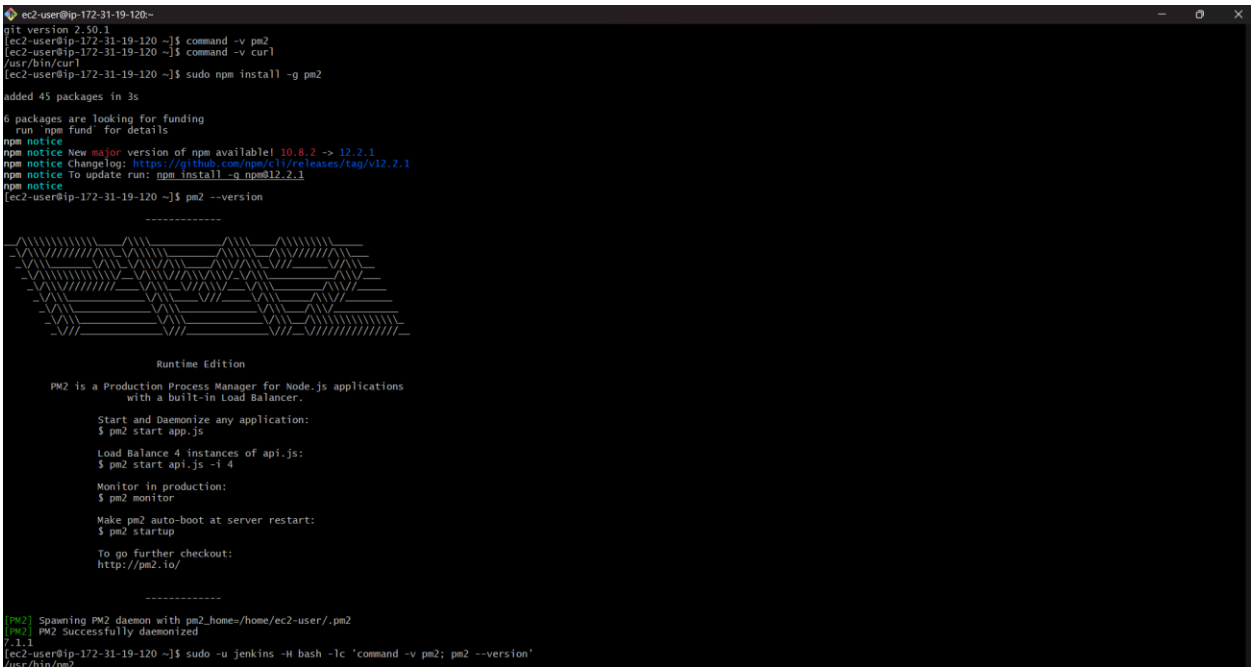
    post {
        success {
            echo 'Asif-Trading-UI CI/CD pipeline completed
successfully.'
            echo 'Application port: 6558'
        }

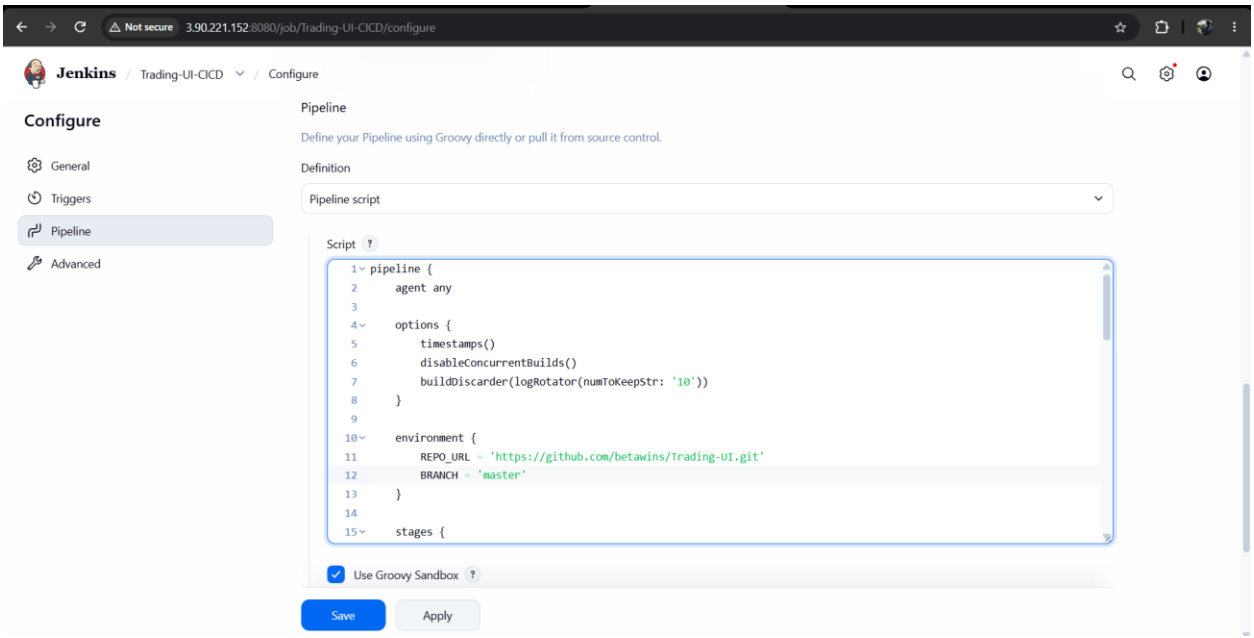
        failure {
            echo 'Pipeline failed. Check the stage logs in Console
Output.'
        }
```

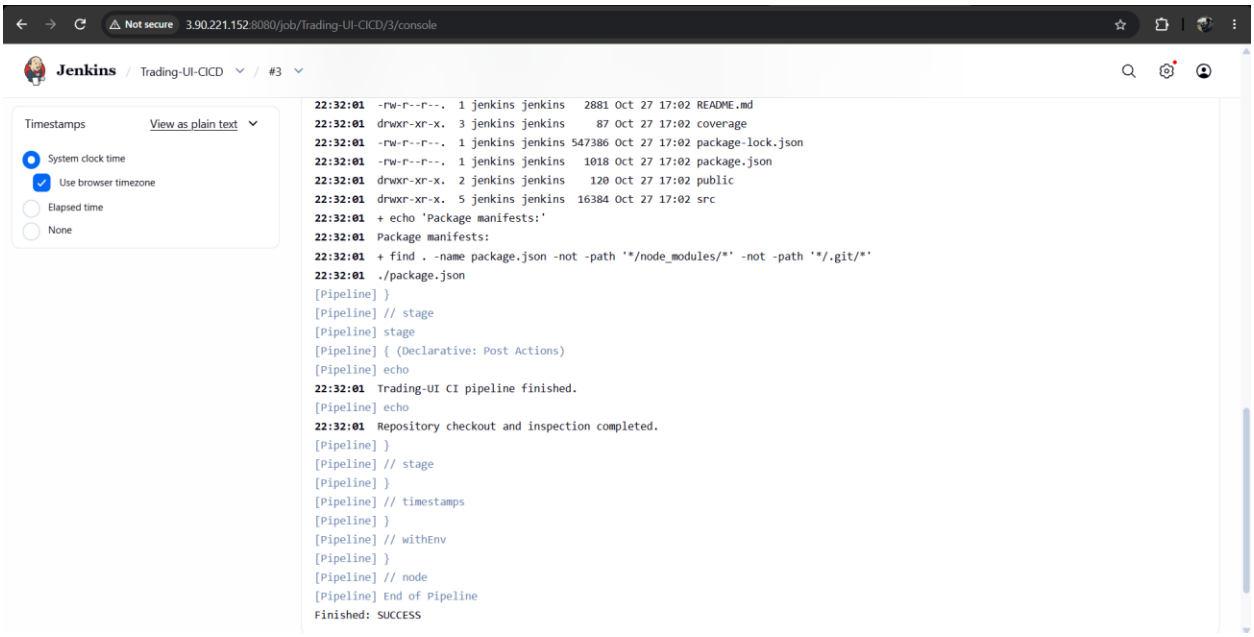
```
        always {
            echo 'Pipeline execution finished.'
        }
    }
}
```



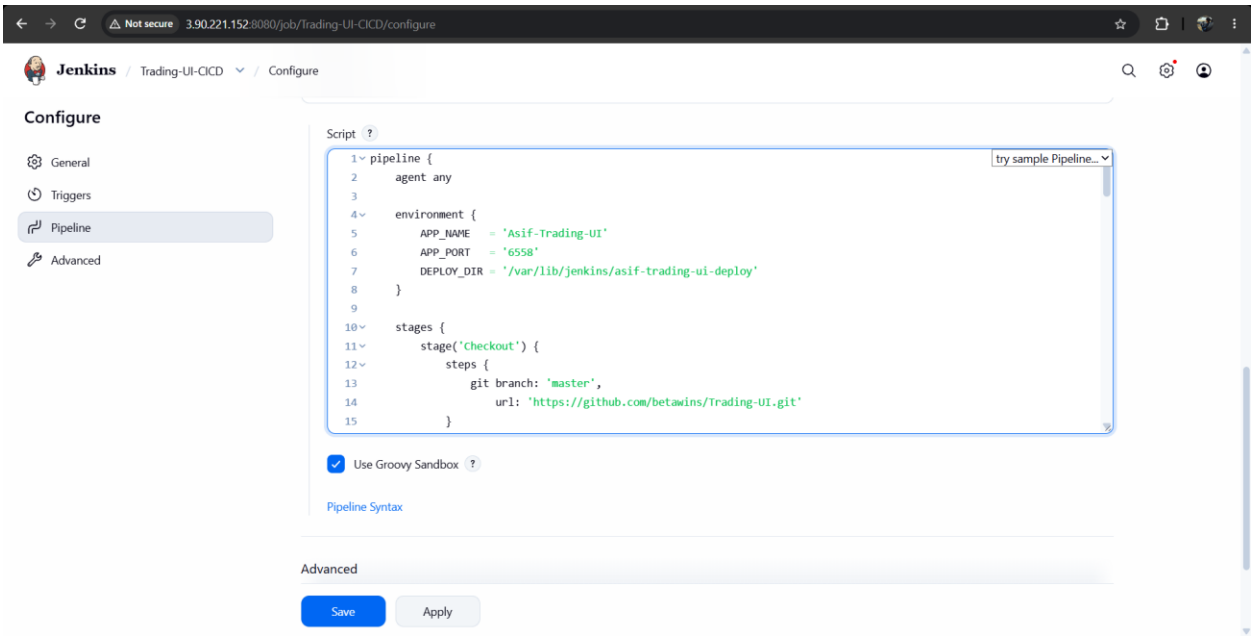


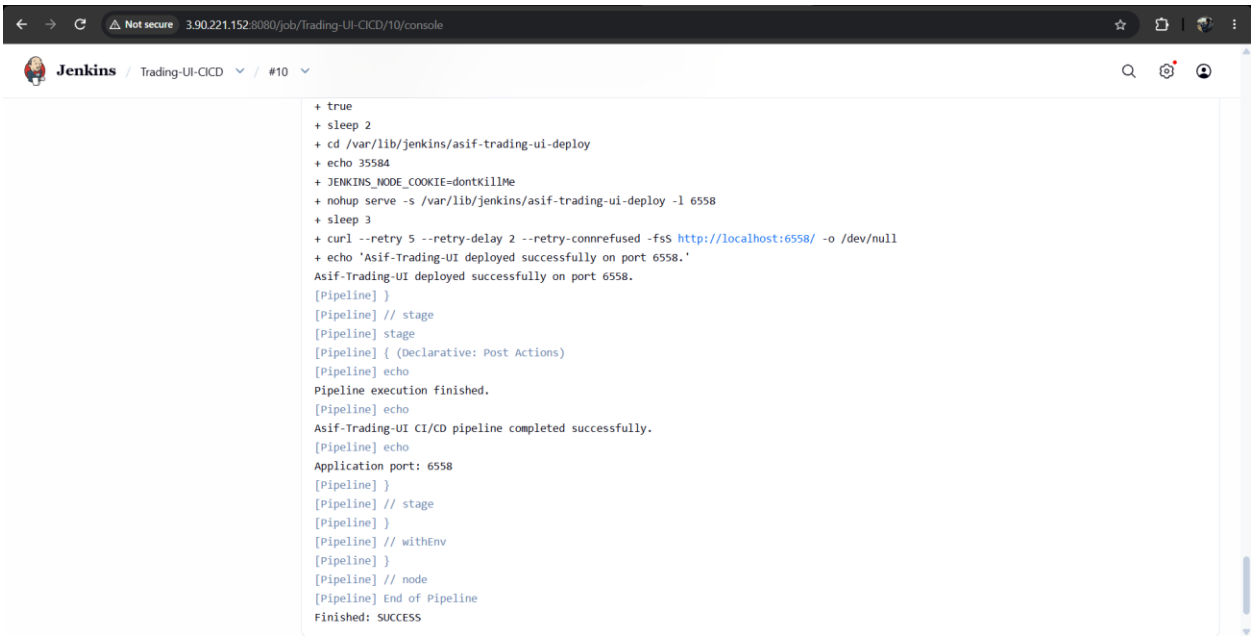

ec2-user@ip-172-31-19-120:~
git version 2.50.1
[ec2-user@ip-172-31-19-120 ~]$ command -v pm2
[ec2-user@ip-172-31-19-120 ~]$ command -v curl
/usr/bin/curl
[ec2-user@ip-172-31-19-120 ~]$ sudo npm install -g pm2
added 45 packages in 3s
6 packages are looking for funding
run `npm fund` for details
npm notice
npm notice New major version of npm available! 10.8.2 -> 12.2.1
npm notice Changelog: https://github.com/npm/cli/releases/tag/v12.2.1
npm notice To update run: npm install -g npm@12.2.1
npm notice
[ec2-user@ip-172-31-19-120 ~]$ pm2 --version
Runtime Edition
PM2 is a Production Process Manager for Node.js applications
with a built-in Load Balancer.
Start and Daemonize any application:
$ pm2 start app.js
Load Balance 4 instances of api.js:
$ pm2 start api.js -i 4
Monitor in production:
$ pm2 monitor
Make pm2 auto-boot at server restart:
$ pm2 startup
To go further checkout:
http://pm2.io/
[PM2] Spawning PM2 daemon with pm2_home=/home/ec2-user/.pm2
[PM2] PM2 Successfully daemonized
7.1.1
[ec2-user@ip-172-31-19-120 ~]$ sudo -u jenkins -H bash -lc 'command -v pm2; pm2 --version'
/usr/bin/pm2

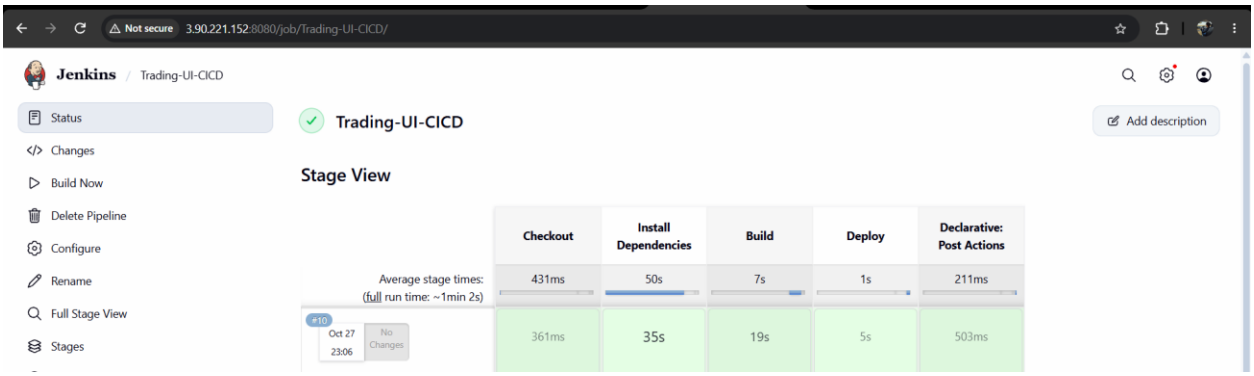

Not secure 3.90.221.152:8080/job/Trading-UI-CICD/configure
Jenkins / Trading-UI-CICD / Configure
Configure
General
Triggers
Pipeline
Advanced
Pipeline
Define your Pipeline using Groovy directly or pull it from source control.
Definition
Pipeline script
Script
pipeline {
agent any
options {
timestamps()
disableConcurrentBuilds()
buildDiscarder(logRotator(numToKeepStr: '10'))
}
environment {
REPO_URL = 'https://github.com/betawins/Trading-UI.git'
BRANCH = 'master'
}
stages {
Use Groovy Sandbox
Save
Apply

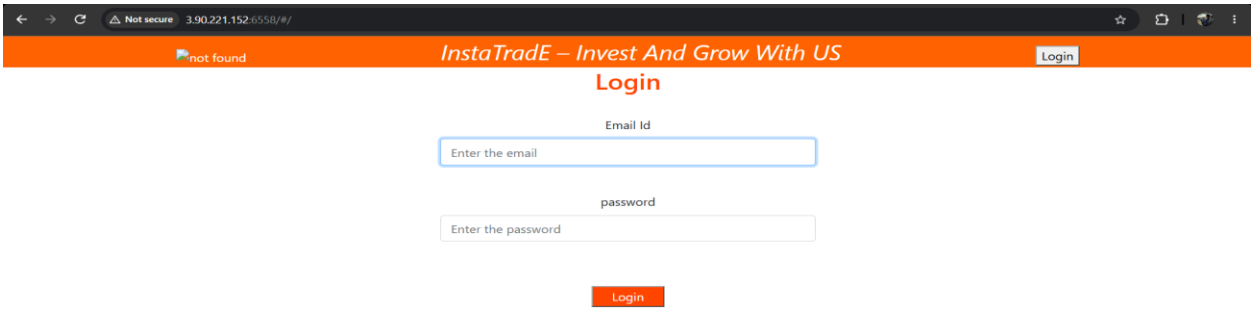

Not secure 3.90.221.152:8080/job/Trading-UI-CICD/3/console
Jenkins / Trading-UI-CICD / #3
Timestamps View as plain text
System clock time
Use browser timezone
Elapsed time
None
22:32:01 -rw-r--r--. 1 jenkins jenkins 2881 Oct 27 17:02 README.md
22:32:01 drwxr-xr-x. 3 jenkins jenkins 87 Oct 27 17:02 coverage
22:32:01 -rw-r--r--. 1 jenkins jenkins 547386 Oct 27 17:02 package-lock.json
22:32:01 -rw-r--r--. 1 jenkins jenkins 1018 Oct 27 17:02 package.json
22:32:01 drwxr-xr-x. 2 jenkins jenkins 120 Oct 27 17:02 public
22:32:01 drwxr-xr-x. 5 jenkins jenkins 16384 Oct 27 17:02 src
22:32:01 + echo 'Package manifests:'
22:32:01 Package manifests:
22:32:01 + find . -name package.json -not -path '*/node_modules/*' -not -path '*/.git/*'
22:32:01 ./package.json
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
22:32:01 Trading-UI CI pipeline finished.
[Pipeline] echo
22:32:01 Repository checkout and inspection completed.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // timestamps
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS


Not secure 3.90.221.152:8080/job/Trading-UI-CICD/
Jenkins / Trading-UI-CICD
Status
Changes
Build Now
Delete Pipeline
Configure
Rename
Full Stage View
Stages
Trading-UI-CICD
Add description
Stage View
Checkout Source
Inspect Project
Declarative: Post Actions
Average stage times: (full run time: ~1s)
571ms
226ms
74ms
#3
Oct 27 22:31
No Changes
404ms
296ms
67ms


Not secure 3.90.221.152:8080/job/Trading-UI-CICD/configure
Jenkins / Trading-UI-CICD / Configure
Configure
General
Triggers
Pipeline
Advanced
Script
try sample Pipeline...
pipeline {
agent any
environment {
APP_NAME = 'Asif-Trading-UI'
APP_PORT = '6558'
DEPLOY_DIR = '/var/lib/jenkins/asif-trading-ui-deploy'
}
stages {
stage('Checkout') {
steps {
git branch: 'master',
url: 'https://github.com/betawins/Trading-UI.git'
}
Use Groovy Sandbox
Pipeline Syntax
Advanced
Save
Apply


Not secure 3.90.221.152:8080/job/Trading-UI-CICD/10/console
Jenkins / Trading-UI-CICD / #10
+ true
+ sleep 2
+ cd /var/lib/jenkins/asif-trading-ui-deploy
+ echo 35584
+ JENKINS_NODE_COOKIE=dontKillMe
+ nohup serve -s /var/lib/jenkins/asif-trading-ui-deploy -l 6558
+ sleep 3
+ curl --retry 5 --retry-delay 2 --retry-connrefused -fsS http://localhost:6558/ -o /dev/null
+ echo 'Asif-Trading-UI deployed successfully on port 6558.'
Asif-Trading-UI deployed successfully on port 6558.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline execution finished.
[Pipeline] echo
Asif-Trading-UI CI/CD pipeline completed successfully.
[Pipeline] echo
Application port: 6558
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS


Not secure 3.90.221.152:8080/job/Trading-UI-CICD/
Jenkins / Trading-UI-CICD
Status
Changes
Build Now
Delete Pipeline
Configure
Rename
Full Stage View
Stages
Trading-UI-CICD
Add description
Stage View
Checkout
Install Dependencies
Build
Deploy
Declarative: Post Actions
Average stage times:
(full run time: ~1min 2s)
431ms
50s
7s
1s
211ms
#10
Oct 27
23:06
No Changes
361ms
35s
19s
5s
503ms


Not secure 3.90.221.152:6558/#/
not found
InstaTradE – Invest And Grow With US
Login
Login
Email Id
Enter the email
password
Enter the password
Login

5. Observation

- Jenkins successfully checked out the application source code from GitHub.
- Dependency installation completed successfully.
- The RxJS module-loading issue was resolved using the version override.
- The OpenSSL compatibility setting allowed the React production build to proceed.
- The production build was copied to the deployment directory.
- The `serve` process started on port `6558`.
- The `curl` verification completed successfully.
- Jenkins reported `Finished: SUCCESS`.
- The React application opened successfully in the browser.

6. Conclusion

The Jenkins CI/CD pipeline was successfully implemented for the Asif-Trading-UI React application. The pipeline automates source-code checkout, dependency installation, production build creation, and deployment on an AWS EC2 instance. The application was hosted on TCP port `6558` and verified through both a local HTTP check and browser access.

This exercise demonstrates how Jenkins can automate the deployment of a frontend application and reduce manual deployment steps.

Task 5: Explain 10 Maven commands.

1. `mvn clean`

Purpose: Removes previously generated build files, usually from the `target/` directory.

```
mvn clean
```

Real-time example: Clean old build output before starting a fresh build.

2. `mvn compile`

Purpose: Compiles the application's main source code.

```
mvn compile
```

Real-time example: Checks whether Java source code compiles successfully.

3. `mvn test`

Purpose: Runs unit tests.

```
mvn test
```

Real-time example: Jenkins checks whether application logic passes its tests before packaging.

4. `mvn package`

Purpose: Compiles the code, runs tests, and packages the application into a JAR or WAR file according to the project configuration.

```
mvn package
```

Real-time example: Generates `target/hiring.war` for deployment to Tomcat, if the project is configured as a WAR.

5. `mvn install`

Purpose: Builds and packages the project, runs tests, and installs the artifact into the local Maven repository.

```
mvn install
```

Real-time example: Makes a locally built library available to other projects on the same machine.

6. `mvn verify`

Purpose: Runs the build lifecycle through verification, including earlier phases and any configured verification checks.

```
mvn verify
```

Real-time example: Runs tests and configured quality or integration checks before the artifact is accepted.

7. `mvn deploy`

Purpose: Publishes the built artifact to a configured remote Maven repository.

```
mvn deploy
```

Real-time example: Uploads a versioned JAR to a repository such as Nexus or Artifactory so other teams can use it.

Note: Maven deploy is not the same as deploying a web application to an EC2 server or Tomcat.

8. `mvn validate`

Purpose: Checks whether the project is correctly configured and its required information is available.

```
mvn validate
```

Real-time example: Detects some project configuration issues before compilation begins.

9. `mvn dependency:tree`

Purpose: Displays the project's dependencies and their relationships.

```
mvn dependency:tree
```

Real-time example: Helps troubleshoot dependency conflicts when multiple libraries require different versions of the same package.

10. `mvn -version`

Purpose: Displays the installed Maven version and the Java runtime Maven uses.

```
mvn -version
```

Real-time example: Checks whether Maven is installed correctly and helps diagnose Java/Maven environment problems on a Jenkins agent.